AI LAB WEEK 4

```
import random

N = 4

def heuristic(state):
    h = 0

    for i in range(N):
        for j in range(i + 1, N):
            # Same row or same diagonal
            if state[i] == state[j] or abs(state[i] - state[j]) ==
abs(i - j):
                h += 1

    return h


def print_board(state):
    for row in range(N):
        for col in range(N):
            if state[col] == row:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()


def get_neighbors(state):
    neighbors = []

    for col in range(N):
        for row in range(N):
            if row != state[col]:
                neighbor = state.copy()
                neighbor[col] = row
                neighbors.append(neighbor)

    return neighbors


def hill_climbing():
    # Random initial state: one queen per column
    current = [random.randint(0, N - 1) for _ in range(N)]

    print("Initial state:", current)
    print("Initial cost:", heuristic(current))

    while True:
        current_cost = heuristic(current)

        # Goal test
        if current_cost == 0:
            break
```

```
        # Generate all neighbors
        neighbors = get_neighbors(current)

        # Select the neighbor with the lowest cost
        best = min(neighbors, key=heuristic)
        best_cost = heuristic(best)

        # Move only if the cost decreases
        if best_cost < current_cost:
            current = best
        else:
            break

    print("\nFinal state:", current)
    print("Final cost:", heuristic(current))

    if heuristic(current) == 0:
        print("Solution found!")
    else:
        print("Hill climbing failed. Restart with a new random state.")

    print("\nFinal chessboard:")
    print_board(current)


hill_climbing()
```

```
Initial state: [1, 1, 0, 0]
Initial cost: 3

Final state: [1, 3, 0, 2]
Final cost: 0
Solution found!

Final chessboard:
. . Q .
Q . . .
. . . Q
. Q . .
```